a. Name of Two GenAI Programs

1. **OpenAI ChatGPT** (GPT-4o model)
2. **Google Gemini** (Gemini 2.5 Pro model) (Note: Per assignment instructions, CoPilot was not paired with ChatGPT as they share the same underlying model family.)

b. How Each GenAI Program Was Accessed

1. **OpenAI ChatGPT:** Accessed via web browser at https://chatgpt.com using a standard user account.
2. **Google Gemini:** Accessed via web browser at https://gemini.google.com using a university/personal Google account.

c. Prompt Used (Identical for Both GenAIs)

"Write a single-file C++ object-oriented program (using classes and objects, not standalone functions) that solves 9x9 Sudoku puzzles using a recursive depth-first search (backtracking) algorithm. The program must read five puzzle files named puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, and puzzle5.txt. In each file, digits 1-9 represent filled cells and underscores '_' represent blank cells. For each puzzle file, the program must output: (1) the puzzle file name, (2) the initial puzzle grid, and (3) all valid solutions to the puzzle, or 'No solution found' if no solution exists."

d. Complete Raw Code Generated by Both GenAIs

Raw Code 1: OpenAI ChatGPT

C++

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>


class Sudoku {
private:
   int board[9][9];


   bool isValid(int r, int c, int val) {
```

```
        for (int i = 0; i < 9; i++) {
            if (board[r][i] == val || board[i][c] == val)
                return false;
        }
        int startRow = (r / 3) * 3;
        int startCol = (c / 3) * 3;
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                if (board[startRow + i][startCol + j] == val)
                    return false;
            }
        }
        return true;
    }

    bool findEmpty(int &r, int &c) {
        for (r = 0; r < 9; r++) {
            for (c = 0; c < 9; c++) {
                if (board[r][c] == 0)
                    return true;
            }
        }
        return false;
    }

public:
    bool load(const std::string& filename) {
        std::ifstream file(filename);
```

```
    if (!file) return false;
    char ch;
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            file >> ch;
            board[i][j] = (ch == '_') ? 0 : (ch - '0');
        }
    }
    return true;
}

void print() {
    for (int i = 0; i < 9; i++) {
        for (int j = 0; j < 9; j++) {
            if (board[i][j] == 0) std::cout << "_ ";
            else std::cout << board[i][j] << " ";
        }
        std::cout << "\n";
    }
}

bool solve() {
    int r, c;
    if (!findEmpty(r, c))
        return true;

    for (int num = 1; num <= 9; num++) {
        if (isValid(r, c, num)) {
```

```
            board[r][c] = num;
            if (solve())
                return true;
            board[r][c] = 0;
        }
      }
      return false;
   }
};

int main() {
   std::string files[] = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};
   for (const auto& f : files) {
      Sudoku s;
      if (s.load(f)) {
         std::cout << "File: " << f << "\nInitial Puzzle:\n";
         s.print();
         if (s.solve()) {
            std::cout << "Solution:\n";
            s.print();
         } else {
            std::cout << "No solution found\n";
         }
         std::cout << "\n";
      }
   }
   return 0;
}
```

Raw Code 2: Google Gemini

C++

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuSolver {
private:
    int grid[9][9];
    int solutionsFound;

    bool isSafe(int row, int col, int num) const {
        for (int i = 0; i < 9; ++i) {
            if (grid[row][i] == num || grid[i][col] == num) {
                return false;
            }
            int boxRow = 3 * (row / 3) + i / 3;
            int boxCol = 3 * (col / 3) + i % 3;
            if (grid[boxRow][boxCol] == num) {
                return false;
            }
        }
        return true;
    }

    void backtrack(int row, int col) {
        if (row == 9) {
```

```
            solutionsFound++;
            std::cout << "Solution " << solutionsFound << ":\n";
            printGrid();
            return;
        }

        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        if (grid[row][col] != 0) {
            backtrack(nextRow, nextCol);
            return;
        }

        for (int num = 1; num <= 9; ++num) {
            if (isSafe(row, col, num)) {
                grid[row][col] = num;
                backtrack(nextRow, nextCol);
                grid[row][col] = 0;
            }
        }
    }

public:
    SudokuSolver() : solutionsFound(0) {}

    bool loadPuzzle(const std::string& filename) {
        std::ifstream in(filename);
```

```
    if (!in.is_open()) return false;
    solutionsFound = 0;
    char c;
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            in >> c;
            grid[i][j] = (c == '_') ? 0 : (c - '0');
        }
    }
    return true;
}

void printGrid() const {
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            std::cout << (grid[i][j] == 0 ? '_' : static_cast<char>('0' + grid[i][j])) << " ";
        }
        std::cout << "\n";
    }
}

void solveFile(const std::string& filename) {
    if (!loadPuzzle(filename)) {
        std::cout << "Could not open " << filename << "\n";
        return;
    }
    std::cout << "Puzzle File: " << filename << "\n";
    std::cout << "Initial Puzzle:\n";
```

```
        printGrid();
        backtrack(0, 0);
        if (solutionsFound == 0) {
            std::cout << "No solution found\n";
        }
        std::cout << "\n";
    }
};

int main() {
    std::vector<std::string> files = {
        "puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"
    };
    SudokuSolver solver;
    for (const auto& file : files) {
        solver.solveFile(file);
    }
    return 0;
}
```

e. Code Analysis Across Lecture Metrics

Metric	OpenAI ChatGPT Code	Google Gemini Code
Correctness	**Partially Incorrect.** While it solves unique puzzles (puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle5.txt) and detects unsolvable puzzles (puzzle4.txt), its solve() method returns true immediately upon finding the	**Fully Correct.** Rather than terminating early on the first base case (row == 9), backtrack(int row, int col) increments solutionsFound, prints the current board, and returns to continue backtracking across all remaining candidate

Metric	OpenAI ChatGPT Code	Google Gemini Code
	first solution (if (solve()) return true;). This violates the requirement to print **all** solutions when a puzzle has multiple solutions.	branches. It also correctly prints "No solution found" when solutionsFound == 0.
Execution Time	$O(9^k)$ **with higher constant overhead.** At every recursive step, findEmpty(r, c) scans the 9x9 grid from (0,0) to locate the next empty cell (up to 81 iterations per node), plus isValid() uses two separate loops (9 iterations + 3×3 nested loop = 18 checks).	$O(9^k)$ **with lower constant overhead.** Passes (row, col) coordinates directly as parameters (nextRow, nextCol), eliminating the $O(81)$ findEmpty() scan at every recursive call. Combines row, column, and 3×3 box checks into a single 9-iteration loop in isSafe().
Space Complexity	$O(k)$ **auxiliary call stack space** (up to 81 stack frames) + $O(1)$ board storage (int board[9][9], 324 bytes). However, main() instantiates a new Sudoku object inside every loop iteration.	$O(1)$ **fixed auxiliary stack depth of 81 frames** (backtrack(0,0) to backtrack(9,0)) + $O(1)$ board storage (int grid[9][9] + int solutionsFound, 328 bytes). Reuses a single SudokuSolver instance across all five files.
Maintainability	**Moderate.** Uses magic numbers, lacks const correctness on read-only methods (isValid, print), lacks defensive input validation in load(), and couples workflow orchestration inside main().	**High.** Uses const member functions (isSafe, printGrid), encapsulates the per-file workflow inside solveFile(), initializes member variables cleanly via constructor initializer lists, and separates recursive state traversal from constraint checking.

f. Selection & Justification

- **Selected Baseline Program: Google Gemini (SudokuSolver)**
- **Justification:**
    1. **Specification Compliance (Correctness):** Gemini's implementation natively satisfies the multi-solution requirement by continuing its depth-first search after reaching the base case (row == 9), whereas ChatGPT's boolean short-circuit (if (solve()) return true;) halts after the first solution.
    2. **Algorithmic Efficiency (Execution Time):** By passing (row, col) coordinates through recursive parameters rather than re-scanning the 81-cell array via findEmpty() on every call, Gemini eliminates up to $81 \times$ per-node scanning overhead and checks all three Sudoku constraints in a single 9-iteration loop.
    3. **Encapsulation & Maintainability:** Gemini encapsulates the complete file-processing lifecycle (loadPuzzle, printGrid, backtrack, solveFile) inside the class and reuses a single object instance in main().

g. Improvements Made to the Selected Baseline Code

1. **Correctness & Input Robustness:**
    - Added explicit grid initialization in the SudokuSolver() constructor so grid[9][9] never contains uninitialized garbage values prior to file loading.
    - Enhanced loadFromFile() to verify that all 81 tokens are successfully extracted (if (!(inFile >> token))) and explicitly validated character ranges ('1' to '9' vs. '_') before closing the file descriptor.
2. **Execution Time:**
    - Replaced the ternary character cast inside printGrid() with direct stream output and avoided trailing spaces at the end of each row (if (c < 8) std::cout << " ";).
3. **Space Complexity:**
    - Retained the fixed 9×9 static array allocation (grid[9][9]) and reused a single SudokuSolver instance across all five puzzle files, keeping heap allocations for the grid at zero ($O(1)$ data space).
4. **Maintainability:**
    - Added comprehensive **Prolog Comments**, **block-level authorship/source summaries**, and **line-by-line In-Line Comments** explaining every statement in the program to meet EECS 348 documentation standards.